BANGLADESH UNIVERSITY OF PROFESSIONALS (BUP)

Faculty of Science & Technology • Department of Computer Science & Engineering

COURSE PROJECT PROPOSAL

NUMERIX: An Interactive Numerical Methods Simulator

A Desktop Application for Step-by-Step Root Finding, Linear System Solving, and Differential Equation Solving in Python

Project Premise: Can a single, easy-to-use desktop application that shows every iteration, graph, error value, and running time help students understand, verify, and compare the numerical methods taught in the Numerical Methods Laboratory?

Submitted By (Student Team)	Submitted To	Course & Academic Details
S M Junaed Ahamed Jimon ID: **25524204141** **Radoan Moktaken Jaman** ID: **25524204085** B.Sc. in Computer Science and Engineering	**Bulbul Ahammad** Assistant Professor CSE, Jahangirnagar University **Farzana Akter** Assistant Professor University of Frontier Technology, Bangladesh	**Course:** Numerical Methods Laboratory **Dept:** CSE, BUP **Project Duration:** 2 Weeks (14 Days)

PRESENTATION SLIDE OUTLINE

The laboratory defense presentation will be delivered in 8–10 minutes following the structured 10-slide roadmap below:

Slide #	Slide Title	Main Content & Visual Artifacts
Slide 1	**Title & Team Introduction**	Project title, BUP CSE credentials, student details, and external faculty advisors.
Slide 2	**Problem Background: Why NUMERIX?**	Hand calculation is slow and error-prone; existing tools show only final answers or need programming.
Slide 3	**Research Question & Objectives**	Can one application implement, visualize, and compare all lab methods in a student-friendly way?
Slide 4	**System Architecture & Tools**	Python core, Tkinter GUI, Matplotlib graphs, NumPy matrices; three modules: Root Finding, Linear Equations, Differential Equations.
Slide 5	**Root-Finding Module**	Bisection, False Position, Newton-Raphson; f(x) input, iteration table with approximate error, convergence plot.
Slide 6	**Linear Equations Module**	Gauss Elimination and Gauss-Jordan; matrix input, row-operation steps, partial pivoting, solution vector.
Slide 7	**Differential Equations Module**	Euler, Modified Euler, and RK4 for dy/dx = f(x, y); step size *h*; solution curves against the exact answer.
Slide 8	**Comparison, Error & Timing**	Side-by-side results, approximate error, number of iterations, and execution time of each method.
Slide 9	**Live Application Demo**	Solve $x^3 - x - 2 = 0$ with three methods; run Euler vs. RK4 on dy/dx = x + y.
Slide 10	**Conclusions & Future Scope**	Summary, limitations, and roadmap: Secant method, LU decomposition, numerical integration, PDF export.

Defense Strategy: Open NUMERIX live, solve $f(x) = x^3 - x - 2$ with Bisection and Newton-Raphson to show the iteration table and graph, then plot Euler vs. RK4 on a differential equation to show how a higher-order method reduces error.

INTRODUCTION & PROBLEM MOTIVATION

Numerical methods are step-by-step procedures that find approximate answers to mathematical problems that are difficult or impossible to solve exactly, such as roots of nonlinear equations, systems of linear equations, and differential equations without closed-form solutions [1] [2]. In the laboratory, students repeat these procedures by hand or in separate scripts, where one small arithmetic slip can spoil a whole result and the idea behind each method is easy to lose.

Existing tools either hide the intermediate steps, cover only some topics, or require programming knowledge. This project develops **NUMERIX**, a desktop application in Python with a Tkinter interface and Matplotlib graphs that brings the lab-syllabus methods together in one place. The user types an equation or matrix, chooses a method, and the application shows every iteration, draws the graph, and reports the approximate error and execution time, so learners can see how each method approaches the answer.

PROJECT OBJECTIVES

- **Goal 1 — Implement the Core Numerical Algorithms:** Code Bisection, False Position, Newton-Raphson, Gauss Elimination, Gauss-Jordan, Euler, Modified Euler, and RK4 in Python as clean, reusable functions that return every intermediate step.
- **Goal 2 — Build an Interactive, Visual Learning Interface:** Create a Tkinter [3] desktop GUI with equation and matrix input, step-by-step iteration tables, and Matplotlib [4] graphs so that each method can be followed visually.
- **Goal 3 — Compare and Validate Methods:** Let the user run several methods on the same problem and compare answers, approximate error, and execution time, and verify results against exact solutions and NumPy [5].

EXPECTED OUTCOMES

- A working NUMERIX desktop application with three sections (Root Finding, Linear Equations, Differential Equations) covering all syllabus methods.
- Verified algorithms: results checked against known exact answers (for example, the root of $x^3 - x - 2 = 0$ at $x \approx 1.52138$) and against NumPy for linear systems.
- Iteration tables and graphs for every method, showing the approximation, approximate error, and stopping condition at each step.
- A comparison panel with execution time and error for each method, together with a short user guide and project report.

RELATED WORKS & NOVELTY ANALYSIS

Related System / Study	Existing Features & Limitations	Project Novelty & Opportunity
Wolfram|Alpha [6]	Online computational engine that solves many maths problems; full step-by-step output is a premium feature and does not follow the specific methods of the lab.	Free and offline; shows every iteration of the exact lab methods, with method-to-method comparison.
MATLAB / GNU Octave [7]	Powerful numerical computing environments with built-in solvers; they need programming knowledge (and a paid licence for MATLAB), and built-in solvers hide the iteration steps.	No coding needed: students type an equation, click Run, and see each iteration and graph.
Symbolab [8]	Online step-by-step solver focused on algebra and calculus; not built around iterative numerical methods or error analysis.	Focuses on approximate (numerical) methods, with iteration tables, error values, and timing.
GeoGebra [9]	Free interactive graphing tool that plots functions but gives no iteration tables and does not compare numerical methods.	Plots the function with each method's iteration path and compares methods side by side.

PROJECT DEVELOPMENT PIPELINE & CONCRETE EXAMPLE

Stage 1 **Input Handling:** The user types f(x), a matrix, or dy/dx = f(x, y) with initial values; the app validates the text and converts it into a safe Python function.

Stage 2 **Method Selection:** Choose a module (Root Finding, Linear Equations, Differential Equations) and a method with its parameters (interval, tolerance, step size *h*).

Stage 3 **Algorithm Execution:** Run the chosen algorithm [10]; store every iteration, row operation, or step, and measure the execution time.

Stage 4 **Visualization:** Show the iteration table and draw Matplotlib graphs: the function with its root, the solution curve, and the error at each iteration.

Stage 5 **Comparison & Report:** Run more methods on the same problem and compare answer, approximate error, and time in one summary panel.

Concrete Example (One Root, Two Methods, One ODE Step): For $f(x) = x^3 - x - 2$ on [1, 2], f(1) = −2 and f(2) = 4, so a root lies inside. Bisection halves the interval each time and needs 10 iterations to narrow it below 0.001, giving x ≈ 1.5210. Newton-Raphson starting at $x_0 = 1.5$ needs only 3 iterations to reach x ≈ 1.52138. For dy/dx = x + y with y(0) = 1 and h = 0.1, one step gives y(0.1) = 1.1000 (Euler), 1.1100 (Modified Euler), and 1.11034 (RK4), while the exact value is 1.11034, so the comparison panel clearly shows RK4 as the most accurate.

EXPECTED FEATURES AND THEIR ACADEMIC JUSTIFICATIONS

Feature	Technical Description	Educational & Practical Justification
Equation & Matrix Input	Text boxes accept f(x) such as x**3 - x - 2 and matrices; input is parsed safely into Python functions.	Lets students try their own lab problems without writing any code.
Eight-Method Solver Library	Bisection, False Position, Newton-Raphson, Gauss Elimination, Gauss-Jordan, Euler, Modified Euler, and RK4 with configurable tolerance and step size *h*.	Directly covers the methods of the Numerical Methods Lab syllabus in one tool.
Step-by-Step Iteration Tables	Table of each iteration, row operation, or ODE step with the approximate error and stopping condition.	Helps students check their hand calculations and understand how each method converges.
Graph Visualization	Matplotlib plots of f(x) with its root, ODE solution curves, and error-versus-iteration charts.	A picture makes ideas such as convergence and step-size error easy to see.
Method Comparison Panel	Runs several methods on the same problem and lists answer, approximate error, and iterations side by side.	Shows why some methods (such as Newton-Raphson and RK4) are faster or more accurate.
Execution Time & Error Report	Measures run time and reports absolute and relative approximate error for each run.	Gives objective, numerical evidence of method quality for laboratory evaluation.

TWO-WEEK TIMELINE & GANTT CHART

Project Activity / Milestone	Week 1 (Days 1–7)							Week 2 (Days 8–14)						
	D1	D2	D3	D4	D5	D6	D7	D8	D9	D10	D11	D12	D13	D14
1. Syllabus Study & Software Design	■	■												
2. Root-Finding Algorithms (3 Methods)		■	■	■										
3. Linear System Solvers (2 Methods)				■	■	■								
4. ODE Solvers (Euler, Modified Euler, RK4)						■	■	■						
5. Verification Against Exact Answers								■	■					
6. Tkinter GUI & Equation Input								■	■	■				
7. Iteration Tables & Matplotlib Graphs										■	■	■		
8. Comparison, Time & Error Panel											■	■		
9. Testing, Bug Fixing & Report Writing												■	■	■
10. Final Project Defense & Demo														★

CONCLUSION

This proposal brings the methods of the Numerical Methods Lab together in one interactive tool. By implementing root-finding, linear-system, and differential-equation solvers with iteration tables, graphs, error values, and timing, NUMERIX turns abstract algorithms into something students can see, test, and compare.

The project strengthens understanding of how numerical methods converge, how step size and iteration count affect accuracy, and how to verify results against exact answers. In the future, NUMERIX can be extended with the Secant method, LU decomposition, Simpson's rule, and exporting results to PDF.

REFERENCES

- **[1]** Burden, R. L., & Faires, J. D. (2010). *Numerical Analysis* (9th ed.). Cengage Learning.
- **[2]** Chapra, S. C., & Canale, R. P. (2015). *Numerical Methods for Engineers* (7th ed.). McGraw-Hill Education.
- **[3]** Python Software Foundation. *tkinter — Python interface to Tcl/Tk.* Python documentation.
- **[4]** Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95.
- **[5]** Harris, C. R., et al. (2020). Array programming with NumPy. *Nature*, 585, 357–362.
- **[6]** Wolfram Research. *Wolfram|Alpha: Computational Intelligence.* wolframalpha.com.
- **[7]** MathWorks. *MATLAB documentation*; GNU Project. *GNU Octave documentation.*
- **[8]** Symbolab. *Online step-by-step math solver.* symbolab.com.
- **[9]** GeoGebra. *Graphing Calculator.* geogebra.org.
- **[10]** National Institute of Standards and Technology (NIST). *Digital Library of Mathematical Functions*, §3.7: Ordinary Differential Equations.